TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero - Junio 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de diseño de software

TÍTULO ACTIVIDAD:

Actividad de valor en equipo - Patrón de diseño Abstract Factory

UNIDAD A EVALUAR:
Unidad 2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:
Casales Crus Josue Jacob 21211927

Andres Gutierrez Cepeda 23211009

Jose Andres Felix Lopez 21211939

Jair Isaac Perez Ricardez 21212024

Bryan Alejandro Rodriguez Gomez 22210345

NOMBRE DEL MAESTRO (A):
Maribel Guerrero Luis

Equipo de desarrollo (Roles)

Andres Gutierrez: Arquitecto

Diseña interfaces y UML.

Jair Pérez: Desarrollador

Implementa fábricas y productos.

Andrés Félix: Integrador

Construye el cliente

Bryan Rodríguez: Tester

Verifica que no haya mezclas indebidas.

Jacob Casales: Documentador

Documenta decisiones.

Asignación de líder: Andrés Félix

Patrón abstract factory

Qué es el patrón utilizado

El patrón de diseño Abstract Factory (Fábrica Abstracta) es un patrón creacional que permite crear familias de objetos relacionados sin especificar sus clases concretas. Su objetivo es que el código pueda trabajar con interfaces en lugar de depender de clases específicas.

Cómo se evita el usar tanto NEW al usar este patrón.

El patrón Abstract Factory evita el uso excesivo de new en el código cliente al delegar la creación de objetos a fábricas especializadas, lo que reduce el acoplamiento y hace que el sistema sea más flexible y fácil de mantener.

Cómo agregar una nueva cocina

Agregar una nueva cocina significa crear una nueva fábrica, lo cual consiste en crear una nueva familia de productos que implemente las mismas interfaces definidas en el sistema. Primero, el sistema ya cuenta con interfaces de productos y una fábrica abstracta que define los métodos para crearlos. Para agregar una nueva fábrica, se deben crear nuevas clases de productos concretos que implementen esas interfaces y después crear una nueva fábrica concreta que implemente la fábrica abstracta y que sea responsable de instanciar esos nuevos productos. De esta manera, el código cliente no necesita modificarse; solo debe utilizar la nueva fábrica cuando se requiera. Esto permite extender el sistema agregando nuevas familias de productos sin alterar las fábricas existentes, cumpliendo con el principio de abierto/cerrado del diseño de software.

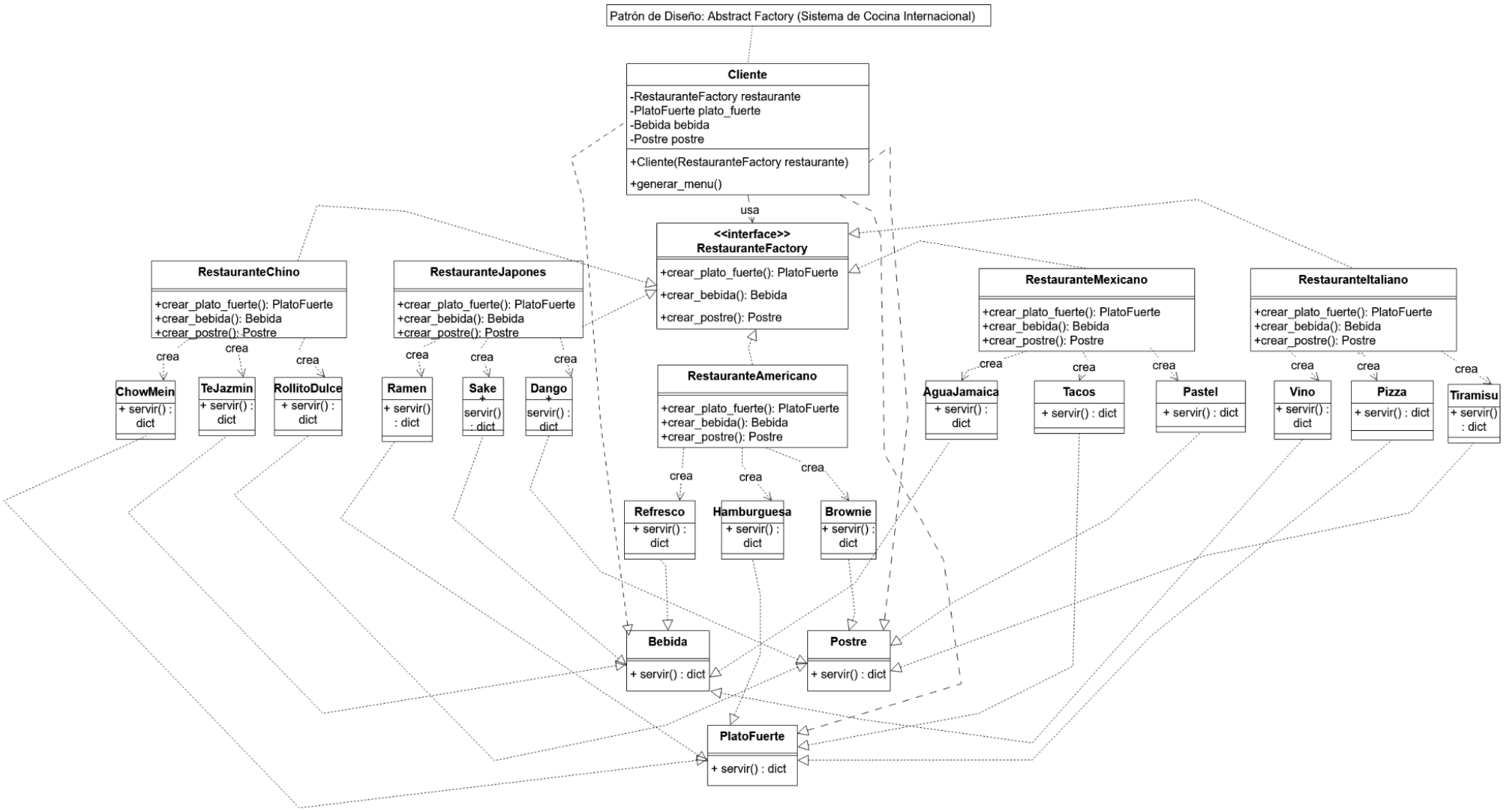
Diagrama de clases UML

El diagrama UML del patrón **Abstract Factory** muestra cómo se organizan las clases para poder crear diferentes familias de objetos sin depender directamente de sus clases concretas.

Primero se define una **interfaz llamada AbstractFactory**, la cual declara los métodos que permitirán crear los distintos tipos de productos. Después existen varias implementaciones de esta fábrica, como **FabricaConcreta1** y **FabricaConcreta2**, que son las encargadas de crear versiones específicas de los productos.

También se definen las interfaces de los productos, como **ProductoA** y **ProductoB**. Estas interfaces permiten que existan diferentes implementaciones, por ejemplo **ProductoA1, ProductoA2, ProductoB1 y ProductoB2**.

El **Cliente** no crea directamente los objetos concretos, sino que trabaja con las interfaces de la fábrica y de los productos. De esta manera el sistema queda desacoplado y se pueden cambiar fácilmente las familias de objetos sin modificar el código del cliente.


Patrón de Diseño: Abstract Factory (Sistema de Cocina Internacional)
Cliente
-RestauranteFactory restaurante
-PlatoFuerte plato_fuerte
-Bebida bebida
-Postre postre
+Cliente(RestauranteFactory restaurante)
+generar_menu()
usa
<<interface>>
RestauranteFactory
+crear_plato_fuerte(): PlatoFuerte
+crear_bebida(): Bebida
+crear_postre(): Postre
RestauranteChino
+crear_plato_fuerte(): PlatoFuerte
+crear_bebida(): Bebida
+crear_postre(): Postre
RestauranteJapones
+crear_plato_fuerte(): PlatoFuerte
+crear_bebida(): Bebida
+crear_postre(): Postre
RestauranteMexicano
+crear_plato_fuerte(): PlatoFuerte
+crear_bebida(): Bebida
+crear_postre(): Postre
RestauranteItaliano
+crear_plato_fuerte(): PlatoFuerte
+crear_bebida(): Bebida
+crear_postre(): Postre
RestauranteAmericano
+crear_plato_fuerte(): PlatoFuerte
+crear_bebida(): Bebida
+crear_postre(): Postre
crea
ChowMein
+ servir() : dict
TeJazmin
+ servir() : dict
RollitoDulce
+ servir() : dict
Ramen
+ servir() : dict
Sake
+ servir() : dict
Dango
+ servir() : dict
AguaJamaica
+ servir() : dict
Tacos
+ servir() : dict
Pastel
+ servir() : dict
Vino
+ servir() : dict
Pizza
+ servir() : dict
Tiramisu
+ servir() : dict
Refresco
+ servir() : dict
Hamburguesa
+ servir() : dict
Brownie
+ servir() : dict
Bebida
+ servir() : dict
Postre
+ servir() : dict
PlatoFuerte
+ servir() : dict

Ventanas de la Aplicación – Implementación del Patrón Abstract Factory

Ventana 1 – Menú de Cocina China

En esta ventana se muestra la selección de la **cocina china** dentro del sistema.
Cuando el usuario elige esta opción, la aplicación utiliza una fábrica concreta que genera los productos correspondientes a este tipo de cocina.

En el menú se presentan tres elementos principales que forman parte de la misma familia de productos:

- **Plato fuerte:** Chow Mein
- **Bebida:** Té de Jazmín
- **Postre:** Rolito dulce con nieve

Todos estos productos pertenecen a la misma categoría cultural y gastronómica, lo cual refleja el objetivo del patrón Abstract Factory: **crear familias de objetos relacionados sin especificar sus clases concretas**.

Ventana 2 – Menú de Cocina Francesa

En esta pantalla se muestra la selección de la **cocina francesa**.
Al elegir esta opción, el sistema cambia la fábrica que genera los productos del menú.

Los elementos que aparecen en este menú son:

- **Plato fuerte:** Ratatouille
- **Bebida:** Champagne
- **Postre:** Macarons

Todos estos productos pertenecen a la misma familia culinaria, lo cual demuestra cómo el patrón permite **mantener consistencia entre los objetos que se crean**.

Ventana 3 – Menú de Cocina Italiana

En esta ventana el usuario selecciona la **cocina italiana**.
Al hacerlo, la aplicación utiliza una fábrica diferente que genera productos propios de esta gastronomía.

El menú incluye:

- **Plato fuerte:** Pasta Carbonara
- **Bebida:** Vino tinto
- **Postre:** Tiramisú

Este ejemplo demuestra cómo el patrón Abstract Factory permite cambiar toda una familia de objetos simplemente cambiando la fábrica que se utiliza.

Ventana 4 – Menú de Cocina Mexicana

En esta ventana se muestra la **cocina mexicana**.
Al seleccionarla, el sistema genera automáticamente los productos correspondientes a esta familia culinaria.

Los elementos que aparecen en el menú son:

- **Plato fuerte:** Tacos de carne asada
- **Bebida:** Agua de jamaica
- **Postre:** Pastel de tres leches

Este ejemplo permite observar claramente cómo cada fábrica concreta produce un conjunto de objetos relacionados que pertenecen a la misma categoría.

Ventana 5 – Menú de Cocina Americana

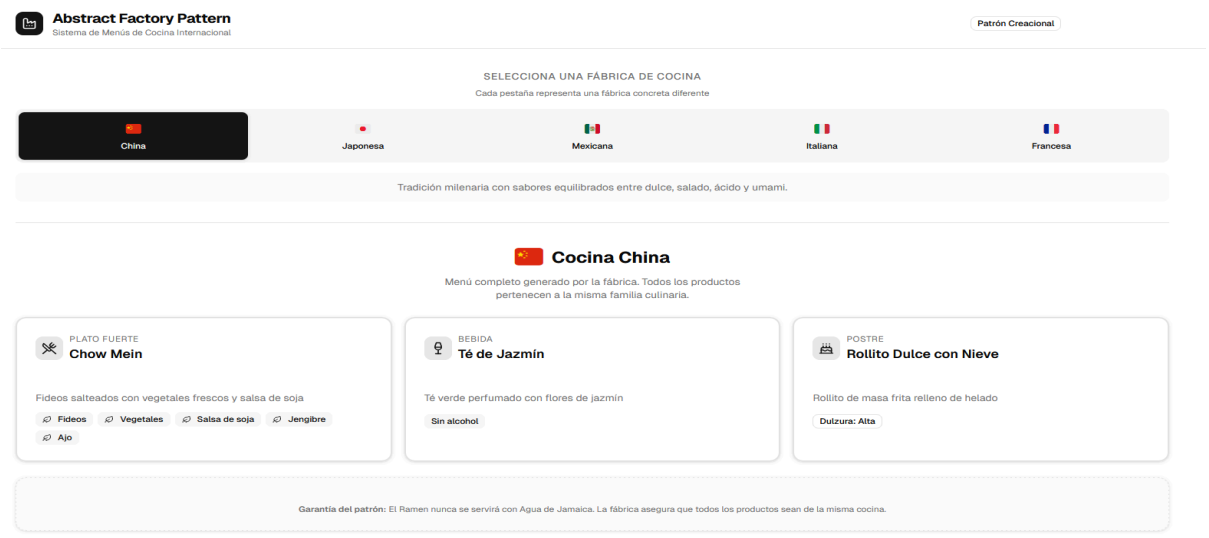
En esta ventana se muestra la **cocina americana**.
Al seleccionarla, el sistema genera automáticamente los productos correspondientes a esta familia culinaria.

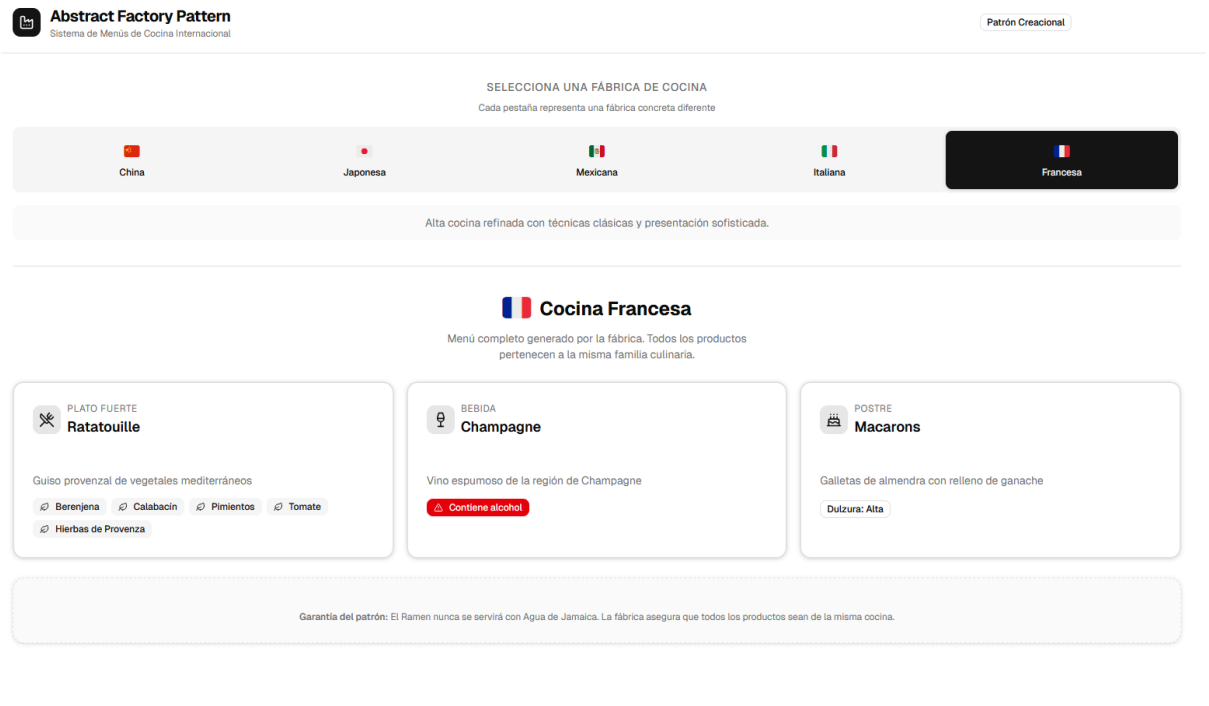
Los elementos que aparecen en el menú son:

- **Plato fuerte:** Hamburguesa
- **Bebida:** Refresco
- **Postre:** Brownie

Este ejemplo permite observar claramente cómo cada fábrica concreta produce un conjunto de objetos relacionados que pertenecen a la misma categoría.

Ventanas


Abstract Factory Pattern
Sistema de Menús de Cocina Internacional
Patrón Creacional
SELECCIONA UNA FÁBRICA DE COCINA
Cada pestaña representa una fábrica concreta diferente
China
Japonesa
Mexicana
Italiana
Francesa
Tradición milenaria con sabores equilibrados entre dulce, salado, ácido y umami.
Cocina China
Menú completo generado por la fábrica. Todos los productos pertenecen a la misma familia culinaria.
PLATO FUERTE
Chow Mein
Fideos salteados con vegetales frescos y salsa de soja
Fideos
Vegetales
Salsa de soja
Jengibre
Ajo
BEBIDA
Té de Jazmín
Té verde perfumado con flores de jazmín
Sin alcohol
POSTRE
Rollito Dulce con Nieve
Rollito de masa frita relleno de helado
Dulzura: Alta
Garantía del patrón: El Ramen nunca se servirá con Agua de Jamaica. La fábrica asegura que todos los productos sean de la misma cocina.


Abstract Factory Pattern
Sistema de Menús de Cocina Internacional
Patrón Creacional
SELECCIONA UNA FÁBRICA DE COCINA
Cada pestaña representa una fábrica concreta diferente
China
Japonesa
Mexicana
Italiana
Francesa
Alta cocina refinada con técnicas clásicas y presentación sofisticada.
Cocina Francesa
Menú completo generado por la fábrica. Todos los productos pertenecen a la misma familia culinaria.
PLATO FUERTE
Ratatouille
Guiso provenzal de vegetales mediterráneos
Berenjena
Calabacín
Pimientos
Tomate
Hierbas de Provenza
BEBIDA
Champagne
Vino espumoso de la región de Champagne
Contiene alcohol
POSTRE
Macarons
Galletas de almendra con relleno de ganache
Dulzura: Alta
Garantía del patrón: El Ramen nunca se servirá con Agua de Jamaica. La fábrica asegura que todos los productos sean de la misma cocina.

Abstract Factory Pattern
Sistema de Menús de Cocina Internacional

Patrón Creacional

SELECCIONA UNA FÁBRICA DE COCINA

Cada pestaña representa una fábrica concreta diferente

China | Japonesa | Mexicana | Italiana | Francesa

Simplicidad elegante donde los ingredientes de calidad son los protagonistas.

Cocina Italiana

Menú completo generado por la fábrica. Todos los productos pertenecen a la misma familia culinaria.

PLATO FUERTE
Pasta Carbonara
Espagueti con salsa cremosa de huevo, queso pecorino y guanciale
Espagueti · Huevos · Pecorino Romano · Guanciale · Pimienta negra

BEBIDA
Vino Tinto
Chianti Classico de la región de Toscana
Contiene alcohol

POSTRE
Tiramisú
Postre de capas de bizcocho, café, mascarpone y cacao
Dulzura: Media

Garantía del patrón: El Ramen nunca se servirá con Agua de Jamaica. La fábrica asegura que todos los productos sean de la misma cocina.

Abstract Factory Pattern
Sistema de Menús de Cocina Internacional

Patrón Creacional

SELECCIONA UNA FÁBRICA DE COCINA

Cada pestaña representa una fábrica concreta diferente

China | Japonesa | Mexicana | Italiana | Francesa

Patrimonio UNESCO con sabores intensos, especias vibrantes y tradición ancestral.

Cocina Mexicana

Menú completo generado por la fábrica. Todos los productos pertenecen a la misma familia culinaria.

PLATO FUERTE
Tacos de Carne Asada
Tortillas de maíz con carne asada, cilantro y cebolla
Tortillas de maíz · Carne asada · Cilantro · Cebolla · Limón · Salsa

BEBIDA
Agua de Jamaica
Bebida refrescante de flor de hibisco
Sin alcohol

POSTRE
Pastel de Tres Leches
Bizcocho empapado en tres tipos de leche con crema batida
Dulzura: Alta

Garantía del patrón: El Ramen nunca se servirá con Agua de Jamaica. La fábrica asegura que todos los productos sean de la misma cocina.

Abstract Factory Pattern

Sistema de Menús de Cocina Internacional

Patrón Creacional

SELECCIONA UNA FÁBRICA DE COCINA

Cada pestaña representa una fábrica concreta diferente

China | Japonesa | Mexicana | Italiana | Francesa

Arte culinario que celebra la frescura, estacionalidad y presentación impecable.

Cocina Japonesa

Menú completo generado por la fábrica. Todos los productos pertenecen a la misma familia culinaria.

PLATO FUERTE

Ramen

Sopa de fideos con caldo de cerdo, huevo y chashu

Fideos ramen · Caldo tonkotsu · Huevo marinado · Chashu · Nori

BEBIDA

Sake

Licor de arroz tradicional japonés servido caliente o frío

Contiene alcohol

POSTRE

Dango

Bolitas de mochi en brocheta con glaseado dulce

Dulzura: Media

Garantía del patrón: El Ramen nunca se servirá con Agua de Jamaica. La fábrica asegura que todos los productos sean de la misma cocina.

Código

PASO 1: PRODUCTOS ABSTRACTOS (Las interfaces o "moldes")

```
class PlatoFuerte(ABC):
    @abstractmethod
    def servir(self) -> dict:
        """Devuelve un diccionario con los datos del plato fuerte"""
        pass

class Bebida(ABC):
    @abstractmethod
    def servir(self) -> dict:
        """Devuelve un diccionario con los datos de la bebida"""
        pass

class Postre(ABC):
    @abstractmethod
    def servir(self) -> dict:
        """Devuelve un diccionario con los datos del postre"""
        pass
```

Paso 1: Productos Abstractos (Interfaces o Moldes)

En esta primera parte del programa se definen los **productos abstractos**, los cuales funcionan como una especie de molde o estructura base que deberán seguir los productos reales que se creen más adelante.

En el código se crean tres clases abstractas: **PlatoFuerte**, **Bebida** y **Postre**. Cada una representa un tipo de producto que puede existir dentro del menú de una cocina. Estas clases no crean objetos directamente, sino que sirven como una guía para que las clases concretas implementen su propio comportamiento.

Cada una de estas clases contiene un método llamado **servir()**, el cual está marcado como **abstractmethod**. Esto significa que cualquier clase que herede de estas clases abstractas está obligada a implementar este método. El propósito de este método es devolver un **diccionario con la información del producto**, como el nombre, descripción o características del platillo.

La razón por la que se utilizan clases abstractas es para mantener una estructura organizada en el sistema. De esta forma todas las bebidas, platos fuertes y postres comparten la misma forma de funcionamiento, aunque cada uno tenga características diferentes dependiendo de la cocina seleccionada.

Este paso es importante dentro del patrón **Abstract Factory**, ya que primero se deben definir los tipos de productos que existirán en el sistema antes de crear las fábricas que los generarán.

PASO 2: PRODUCTOS CONCRETOS (Las implementaciones reales)

```
# --- Familia 1: Comida Japonesa ---

class Ramen(PlatoFuerte):

    def servir(self):

        return {

            "nombre": "Ramen",

            "descripcion": "Fideos tradicionales japoneses en caldo rico en umami con cerdo chashu y huevo.",

            "tags": ["Fideos", "Umami", "Cerdo"]

        }

class Sake(Bebida):

    def servir(self):

        return {

            "nombre": "Sake",

            "descripcion": "Vino de arroz tradicional japonés servido caliente en taza de cerámica.",

            "tags": ["Arroz", "Fermentado", "Con Alcohol"]

        }

class Dango(Postre):

    def servir(self):

        return {

            "nombre": "Dango",

            "descripcion": "Bolitas dulces y masticables de harina de arroz servidas en un palillo de bambú.",

            "tags": ["Arroz", "Dulce", "Tradicional"]

        }
```

```
# --- Familia 2: Comida Mexicana ---

class Tacos(PlatoFuerte):

    def servir(self):

        return {

            "nombre": "Tacos de carne asada",

            "descripcion": "Tortillas de maíz hechas a mano con jugosa carne asada al carbón, guacamole y salsa.",

            "tags": ["Maíz", "Carne", "Picante"]

        }

class AguaJamaica(Bebida):

    def servir(self):

        return {

            "nombre": "Agua de Jamaica",

            "descripcion": "Infusión fría de flor de jamaica, refrescante y ligeramente endulzada.",

            "tags": ["Refrescante", "Frío", "Sin Alcohol"]

        }

class Pastel(Postre):

    def servir(self):

        return {

            "nombre": "Pastel de tres leches",

            "descripcion": "Bizcocho esponjoso bañado en tres tipos de leche, decorado con durazno y crema.",

            "tags": ["Dulzura Alta", "Lácteos", "Esponjoso"]

        }
```

```
# --- Familia 3: Comida China ---

class ChowMein(PlatoFuerte):

    def servir(self):

        return {

            "nombre": "Chow Mein",

            "descripcion": "Fideos salteados al wok con vegetales frescos, pollo y salsa de soja tradicional.",

            "tags": ["Fideos", "Vegetales", "Wok"]

        }

class TeJazmin(Bebida):

    def servir(self):

        return {

            "nombre": "Té Jazmín",

            "descripcion": "Té verde caliente perfumado con delicadas flores de jazmín, ideal para la digestión.",

            "tags": ["Té Verde", "Caliente", "Digestivo"]

        }

class RollitoDulce(Postre):

    def servir(self):

        return {

            "nombre": "Rollito Dulce con nieve",

            "descripcion": "Delicioso rollito de masa frita crujiente relleno de helado artesanal de vainilla.",

            "tags": ["Frito", "Helado", "Dulzura Alta"]

        }
```

```
# --- Familia 4: Comida Italiana ---

class Pizza(PlatoFuerte):

    def servir(self):

        return {

            "nombre": "Pizza Margherita",

                "descripcion": "Masa artesanal al horno de leña con salsa de tomate pomodoro, mozzarella fresca y albahaca.",

            "tags": ["Horno de Leña", "Queso", "Tradicional"]

        }

class Vino(Bebida):

    def servir(self):

        return {

            "nombre": "Vino Tinto",

             "descripcion": "Copa de vino tinto reserva, con notas profundas a frutos rojos y madera.",

            "tags": ["Uva", "Cosecha", "Con Alcohol"]

        }

class Tiramisu(Postre): # Sin acento para evitar errores de compilación

    def servir(self):

        return {

            "nombre": "Tiramisú",

             "descripcion": "Postre frío de capas de bizcocho bañadas en café espresso y suave crema de mascarpone.",

            "tags": ["Café", "Mascarpone", "Frío"]

        }
```

```
# --- Familia 5: Comida Americana ---

class Hamburguesa(PlatoFuerte):

    def servir(self):

        return {

            "nombre": "Hamburguesa Clásica",

                "descripcion": "Carne de res a la parrilla con queso cheddar derretido, lechuga, tomate y papas fritas.",

            "tags": ["Carne", "Queso", "Parrilla"]

        }

class Refresco(Bebida):

    def servir(self):

        return {

            "nombre": "Refresco de cola",

             "descripcion": "Bebida carbonatada dulce y muy fría, servida en vaso grande con abundantes hielos.",

            "tags": ["Burbujas", "Azúcar", "Frío"]

        }

class Brownie(Postre):

    def servir(self):

        return {

            "nombre": "Brownie con helado",

              "descripcion": "Cuadro de pastel de chocolate denso y caliente, servido con una bola de helado de vainilla.",

            "tags": ["Chocolate", "Caliente y Frío", "Dulzura Alta"]

        }
```

Paso 2: Productos Concretos (Implementaciones Reales)

En esta segunda parte del programa se crean los **productos concretos**, es decir, las clases que representan los platillos, bebidas y postres reales que aparecerán en el menú de cada tipo de cocina.

Cada una de estas clases **hereda de las clases abstractas** definidas anteriormente: **PlatoFuerte, Bebida y Postre**. Gracias a esto, todas las clases deben implementar el método **servir()**, el cual devuelve la información del producto en forma de un diccionario con datos como el nombre, la descripción y algunas características o etiquetas.

Dentro del sistema se definieron **cinco familias de productos**, cada una correspondiente a un tipo de cocina diferente:

- Cocina japonesa
- Cocina mexicana
- Cocina china
- Cocina italiana
- Cocina americana

Cada familia contiene tres tipos de productos: un **plato fuerte**, una **bebida** y un **postre**. Por ejemplo, en la cocina japonesa se incluyen el **Ramen** como plato fuerte, **Sake** como bebida y **Dango** como postre. De la misma forma, cada cocina tiene sus propios elementos representativos.

Por ejemplo:

- La **cocina mexicana** incluye tacos de carne asada, agua de jamaica y pastel de tres leches.
- La **cocina china** contiene chow mein, té de jazmín y un rollito dulce con nieve.
- La **cocina italiana** incluye pizza, vino tinto y tiramisú.

- La **cocina americana** presenta una hamburguesa clásica, refresco de cola y brownie con helado.

Cada clase implementa el método **servir()**, que devuelve la información del producto organizada en un diccionario. Esto permite que la aplicación pueda mostrar fácilmente los datos en la interfaz gráfica.

Esta parte es importante dentro del patrón **Abstract Factory**, ya que aquí es donde se crean las **implementaciones reales de los productos**, que después serán generadas por las fábricas concretas dependiendo del tipo de cocina que el usuario seleccione.

PASO 3: FÁBRICA ABSTRACTA (La interfaz creadora principal)

```
class RestauranteFactory(ABC):
    @abstractmethod
    def crear_plato_fuerte(self) -> PlatoFuerte: pass

    @abstractmethod
    def crear_bebida(self) -> Bebida: pass

    @abstractmethod
    def crear_postre(self) -> Postre: pass
```

Paso 3: Fábrica Abstracta (Interfaz Creadora Principal)

En esta parte del código se define la **fábrica abstracta**, la cual funciona como la estructura principal que se encargará de crear los diferentes productos del sistema.

La clase llamada **RestauranteFactory** es una clase abstracta que establece los métodos que todas las fábricas concretas deberán implementar. En otras palabras, esta clase define las reglas que deben seguir las fábricas que crearán los distintos menús de comida.

Dentro de esta clase se declaran tres métodos abstractos:

- **crear_plato_fuerte()**
- **crear_bebida()**
- **crear_postre()**

Cada uno de estos métodos tiene la responsabilidad de generar un tipo específico de producto. Sin embargo, en esta clase no se define cómo se crean esos productos, ya que esa tarea será responsabilidad de las fábricas concretas que se implementarán más adelante.

La ventaja de trabajar de esta manera es que el programa puede utilizar la fábrica abstracta sin preocuparse por los detalles de las clases concretas que realmente se están utilizando. Esto permite que el sistema sea más flexible y fácil de modificar, ya que si se agrega un nuevo tipo de cocina solo será necesario crear una nueva fábrica concreta que implemente estos métodos.

Dentro del patrón **Abstract Factory**, esta clase representa el punto central del diseño, porque define la forma en la que se crearán las familias de productos sin depender directamente de sus implementaciones específicas.

PASO 4: FÁBRICAS CONCRETAS (Los ensambladores de familias)

```
class RestauranteChino(RestauranteFactory):
    def crear_plato_fuerte(self) -> PlatoFuerte: return ChowMein()
    def crear_bebida(self) -> Bebida: return TeJazmin()
    def crear_postre(self) -> Postre: return RollitoDulce()

class RestauranteJapones(RestauranteFactory):
    def crear_plato_fuerte(self) -> PlatoFuerte: return Ramen()
    def crear_bebida(self) -> Bebida: return Sake()
    def crear_postre(self) -> Postre: return Dango()

class RestauranteMexicano(RestauranteFactory):
    def crear_plato_fuerte(self) -> PlatoFuerte: return Tacos()
    def crear_bebida(self) -> Bebida: return AguaJamaica()
    def crear_postre(self) -> Postre: return Pastel()

class RestauranteItaliano(RestauranteFactory):
    def crear_plato_fuerte(self) -> PlatoFuerte: return Pizza()
    def crear_bebida(self) -> Bebida: return Vino()
    def crear_postre(self) -> Postre: return Tiramisu()

class RestauranteAmericano(RestauranteFactory):
    def crear_plato_fuerte(self) -> PlatoFuerte: return Hamburguesa()
    def crear_bebida(self) -> Bebida: return Refresco()
    def crear_postre(self) -> Postre: return Brownie()
```

Paso 4: Fábricas Concretas (Los ensambladores de familias)

En esta parte del programa se implementan las **fábricas concretas**, las cuales son las encargadas de crear los productos reales que pertenecen a cada tipo de cocina.

Cada una de estas fábricas **hereda de la clase abstracta `RestauranteFactory`**, lo que significa que están obligadas a implementar los tres métodos definidos anteriormente: `crear_plato_fuerte()`, `crear_bebida()` y `crear_postre()`.

La función principal de estas clases es **ensamblar una familia completa de productos relacionados**. Es decir, cada fábrica genera un conjunto de platillos que pertenecen a una misma cultura gastronómica.

Por ejemplo:

- **RestauranteChino** crea un menú compuesto por *Chow Mein*, *Té de Jazmín* y *Rollito dulce*.
- **RestauranteJapones** genera *Ramen*, *Sake* y *Dango*.
- **RestauranteMexicano** produce *Tacos*, *Agua de Jamaica* y *Pastel de tres leches*.
- **RestauranteItaliano** crea *Pizza*, *Vino tinto* y *Tiramisú*.
- **RestauranteAmericano** genera *Hamburguesa*, *Refresco* y *Brownie con helado*.

Cada método simplemente regresa una **instancia del producto correspondiente**, lo que permite que el sistema pueda construir un menú completo dependiendo del restaurante seleccionado.

Esta parte es clave dentro del patrón **Abstract Factory**, porque aquí es donde se crean las **familias completas de objetos relacionados**. Gracias a este diseño, el programa puede cambiar de un tipo de cocina a otro simplemente utilizando una fábrica diferente, sin necesidad de modificar el resto del código.

Esto ayuda a mantener el sistema organizado, flexible y fácil de ampliar en el futuro. Por ejemplo, si se quisiera agregar una nueva cocina, como la comida francesa, solo sería necesario crear una nueva fábrica concreta con sus respectivos productos.

PASO 5: EL CÓDIGO CLIENTE (Aislamiento de la lógica)

```
class Cliente:
    def __init__(self, restaurante: RestauranteFactory):
        # A. Recibir la fábrica por constructor
        self.restaurante = restaurante
        self.plato_fuerte = None
        self.bebida = None
        self.postre = None

    def generar_menu(self):
        # B. Generar el menú completo a través de la interfaz abstracta
        self.plato_fuerte = self.restaurante.crear_plato_fuerte()
        self.bebida = self.restaurante.crear_bebida()
        self.postre = self.restaurante.crear_postre()
```

Paso 5: El Código Cliente (Aislamiento de la lógica)

En esta parte del programa se encuentra la clase **Cliente**, que es la encargada de utilizar las fábricas para generar el menú que se mostrará en la aplicación.

El objetivo principal de esta clase es **separar la lógica del programa de las implementaciones concretas**. Es decir, el cliente no necesita saber exactamente qué tipo de productos se están creando, solamente utiliza la interfaz que proporciona la fábrica.

Dentro del constructor (`__init__`) se recibe como parámetro un objeto de tipo **RestauranteFactory**. Esto significa que el cliente puede trabajar con cualquier tipo

de restaurante, ya sea chino, japonés, mexicano, italiano o americano. De esta manera, el cliente no depende directamente de las clases concretas, sino únicamente de la interfaz de la fábrica.

Además, se crean tres atributos que almacenarán los elementos del menú:

- **plato_fuerte**
- **bebida**
- **postre**

Posteriormente, el método **generar_menu()** se encarga de crear el menú completo. Para hacerlo, utiliza los métodos definidos en la fábrica abstracta:

- `crear_plato_fuerte()`
- `crear_bebida()`
- `crear_postre()`

Cada uno de estos métodos devuelve un objeto correspondiente al tipo de cocina seleccionado. De esta forma, el cliente obtiene todos los elementos del menú sin preocuparse por cómo fueron creados.

Esta parte del código demuestra uno de los principales beneficios del patrón **Abstract Factory**, ya que el cliente puede trabajar con diferentes familias de productos sin modificar su lógica interna. Simplemente cambiando la fábrica que se le proporciona, el sistema puede generar un menú completamente diferente.

PASO 6: SERVIDOR WEB FLASK (Endpoints y ruteo)

```
app = Flask(__name__)


# Diccionario que enlaza el texto del HTML con la clase Fábrica correspondiente.
# Esto cumple el punto "D. Cambio de cocina = cambiar la fábrica."
diccionario_fabricas = {
    "mexicana": RestauranteMexicano,
    "japonesa": RestauranteJapones,
    "china": RestauranteChino,
    "italiana": RestauranteItaliano,
    "americana": RestauranteAmericano
}


@app.route('/')
def inicio():
    """Ruta base que renderiza la interfaz HTML de Tailwind"""
    return render_template('index.html')


@app.route('/ordenar', methods=['POST'])
def ordenar():
    """Ruta API que recibe la solicitud de JS y devuelve el menú en JSON"""
    datos = request.get_json()
    tipo_solicitado = datos.get('cocina')
```

```
# Verificamos que la solicitud sea válida
    if tipo_solicitado in diccionario_fabricas:
        # Instanciamos dinámicamente la fábrica correcta elegida por el usuario
        fabrica_seleccionada = diccionario_fabricas[tipo_solicitado]()

        # Le pasamos la fábrica al cliente por su constructor y generamos el menú
        cliente = Cliente(fabrica_seleccionada)
        cliente.generar_menu()

        # C. Servir los platillos sin saber qué cocina está usando.
            # Ejecutamos el método servir() de cada producto abstracto y armamos la respuesta
        respuesta = {
            "plato": cliente.plato_fuerte.servir(),
            "bebida": cliente.bebida.servir(),
            "postre": cliente.postre.servir()
        }

        return jsonify(respuesta)
    else:
        return jsonify({"error": "Tipo de cocina no reconocido"}), 400

# Arranque del servidor de desarrollo
if __name__ == '__main__':
    app.run(debug=True)
```

Paso 6: Servidor Web con Flask (Endpoints y ruteo)

En esta última parte del programa se implementa el **servidor web utilizando Flask**, el cual se encarga de conectar la lógica del sistema con la interfaz que ve el usuario en el navegador.

Primero se crea la aplicación de Flask con la línea `app = Flask(__name__)`. Esto inicializa el servidor que permitirá recibir solicitudes desde la página web y responder con la información correspondiente.

Después se define un **diccionario llamado `diccionario_fabricas`**, el cual relaciona el nombre de cada tipo de cocina con su respectiva clase de fábrica. Gracias a este diccionario, cuando el usuario selecciona una cocina desde la interfaz, el sistema puede identificar qué fábrica debe utilizar para generar el menú.

Por ejemplo, si el usuario selecciona **mexicana**, el sistema utilizará la clase `RestauranteMexicano`. Si selecciona **japonesa**, se utilizará `RestauranteJapones`, y así sucesivamente con las demás opciones disponibles.

Posteriormente se definen dos rutas principales dentro del servidor.

La primera es la ruta **"/"**, que funciona como la página principal de la aplicación. Esta ruta simplemente carga el archivo **index.html**, el cual contiene la interfaz gráfica donde el usuario puede seleccionar el tipo de cocina.

La segunda ruta es **"/ordenar"**, que funciona como una API que recibe solicitudes mediante el método **POST**. Cuando el usuario selecciona una cocina en la página, el navegador envía esa información al servidor en formato JSON.

El servidor recibe el tipo de cocina solicitado y verifica si existe dentro del diccionario de fábricas. Si la opción es válida, el sistema crea una instancia de la fábrica correspondiente y la envía al **Cliente**. Después se ejecuta el método `generar_menu()`, el cual crea los tres elementos del menú: el plato fuerte, la bebida y el postre.

Finalmente, el servidor ejecuta el método **servir()** de cada producto y construye una respuesta en formato **JSON**, que contiene toda la información de los platillos. Esta respuesta es enviada nuevamente al navegador para que la interfaz pueda mostrar el menú seleccionado.

Si el usuario solicita una cocina que no existe, el servidor devuelve un mensaje de error indicando que el tipo de cocina no es reconocido.

Para finalizar, el programa incluye la sección que inicia el servidor de desarrollo de Flask. Esto permite ejecutar la aplicación y probarla localmente en el navegador mientras se está desarrollando el proyecto.

Con esta parte se completa el funcionamiento del sistema, ya que conecta el patrón **Abstract Factory**, la lógica del programa y la interfaz web que interactúa con el usuario.

Referencias

- Abstract Factory. (s. f.). https://refactoring.guru/es/design-patterns/abstract-factory
- Design Patterns and Refactoring. (s. f.). https://sourcemaking.com/design_patterns/abstract_factory
- Abstract Factory. (s. f.-b). https://reactiveprogramming.io/blog/es/patrones-de-diseno/abstract-factory
- Caballero, C. (2021, 28 febrero). Understanding Design Patterns: Abstract Factory. DEV Community. https://dev.to/carlillo/understanding-design-patterns-abstract-factory-23e7